Synthesis Comparison Report

This report compares four papers from the Hyperon/PRIMUS research program that together form one evolving line of work on AGI motivation: OpenPsi (2013), the MetaMo theory paper (AGI-25, 2025), the MetaMo-to-OpenPsi bridge paper (2025), and SubRep (September 2025), a 155-page preliminary draft currently in R&D.

The four papers answer different stages of the same question. OpenPsi shows how emotion and motivation can emerge from internal demand dynamics, and it remains the only experimentally tested system in the set. MetaMo gives the abstract mathematical shape any stable motivation system should have, using an appraisal comonad, a decision monad, and contraction-based stability. The bridge paper makes MetaMo concrete by wiring OpenPsi and MAGUS into its slots and adding the individuation and transcendence overgoals. SubRep then takes the output of all that, a set of current motives, and solves the next problem down: decomposing motives into sub-motives through learning rather than hand design, certifying candidate skills against a whole region of plausible motives instead of a single point, and composing those certified skills in an algebra where safety guarantees survive composition.

The comparison runs across four dimensions. Section 1 covers motivational state, from OpenPsi's demands to MetaMo's $G \times M$to SubRep's motive slice. Section 2 covers the core operators, the pseudo-bimonad versus the residuated quantale. Section 3 covers who decides the structure of motivation, hand-designed overgoals versus the learned MDN, along with stability. Section 4 covers execution engines, cross-paradigm learning through EASA, Hyperon and MORK integration, attention economics, and an honest side-by-side of guarantees and limitations.

The single most important thread connecting the papers is an inversion: constraints that MetaMo imposed as axioms, such as moderated goal change, become in SubRep motives the agent learns to decompose and pursue on its own. What the whole stack still lacks, from 2013 to today, is a second experiment.

1. Orientation: Where SubRep Sits in the Lineage

OpenPsi (2013) is the oldest and the most empirical paper. It asks how emotion and motivation can emerge from internal demand dynamics, and it actually tested this by running a virtual robot in a Minecraft-style world. The MetaMo theory paper sits at the opposite extreme. It is pure abstract mathematics, using category theory, functional analysis, and topology to define what shape any good motivation system must have. The bridge paper connects these two levels by filling MetaMo's abstract slots with concrete machinery: OpenPsi becomes the appraisal comonad, MAGUS becomes the decision monad, and the two overgoals, individuation and transcendence, sit on top of everything.

SubRep is the next layer of this stack, and it answers a question the other three papers mostly leave open. MetaMo and the bridge paper can tell you what the agent wants right now, but they say very little about how a high-level goal gets broken into concrete, reusable sub-behaviors. SubRep is exactly that missing piece. It is a framework for recursive subgoal decomposition and for deciding which candidate skills, called options, are safe and useful enough to be added to the agent's permanent library. The paper explicitly cites MetaMo and describes itself as the planner-facing layer that consumes MetaMo's motive outputs. So the full dependency chain now reads: OpenPsi supplies the feel step, MAGUS with the overgoals supplies the choose step, and SubRep supplies the decompose-and-certify step that turns chosen goals into executable, composable behavior.

1.2 SubRep's Starting Point: Sutton's Reward-Respecting Subtasks

An interesting fact about SubRep is that it does not begin from the OpenCog tradition at all. It begins from mainstream reinforcement learning, specifically from Sutton et al.'s 2023 work on reward-respecting subtasks. Goertzel himself credits Sutton's keynote at AGI-25 for starting this line of thinking.

The background problem is option discovery in hierarchical RL. An option is a policy with its own stopping rule, and the space of possible options is enormous. Most discovery methods, such as bottleneck detection or eigenoptions, end up picking subtasks that ignore the main task's reward. Sutton's solution was to keep discovered options tied to the main reward, which made temporal abstraction practical inside model-based RL.

Goertzel's critique is that this still tunes each option to one reward weighting at a time. For real agents facing uncertain, multi-objective, and shifting priorities, this is too fragile. The better move is to decompose top-level rewards and goals into collections of sub-motives, and then admit options that are robustly useful across an adaptively constructed set of plausible motives, not just one weight vector. This is the same complaint MetaMo made against scalar reward maximization, which it called scalar collapse, but now the complaint is made operational at the planning level.

1.3 The Two Admission Rules: CDS and PDS

SubRep's core move is to replace the question "does this option help the reward" with "does this option robustly help across a whole region of plausible motive weightings." Every option oexports an expectation-model summary $(\hat{r}, \hat{n})$, meaning its expected accumulated reward and its expected next-state features. This summary turns the option into a decision transformer, as described in the next section. The planner's baseline at any state is the best it can already do using its current admitted library B:

$$B_{base}(x; w) = \max_{b \in B} (\hat{r}(x, b) + w^T \hat{n}(x, b))$$

The first admission rule is Cone-Dominant Subtasks, or CDS. It admits an option only if the option beats the baseline uniformly over an entire cone Wof plausible motive weights. The admission margin is the worst-case improvement:

$$m_o(x) = \inf_{w \in W} (B_o(x; w) - B_{base}(x; w))$$

The option is admitted if this worst-case margin is nonnegative, either in expectation over the planning distribution or with high probability. CDS is the conservative, safety-first rule. The option must help no matter which direction inside the cone the agent's priorities lean.

The second rule is Pareto-Dominant Subtasks, or PDS, and it is deliberately weaker. Instead of a continuous cone, it uses a finite cover $W_{ref} = \{w^{(1)}, \dots, w^{(K)}\}$of reference weight vectors and compares componentwise. For each reference ray, the component gap is:

$$\Delta_k(x, o) = B_o(x; w^{(k)}) - B_{base}(x; w^{(k)})$$

The budgeted PDS rule then admits option oif:

$$| \{k: \Delta_k(x, o) \geq \varepsilon\} | \geq qK, \Delta_k(x, o) \geq -\rho \quad \forall k$$

Here $\varepsilon > 0$is the significant-improvement threshold, $q \in (0,1]$is the coverage fraction, and $\rho \geq 0$is the downside budget. In plain words, the option must clearly improve things on at least a fraction qof the reference rays, and it is never allowed to lose more than ρon any ray.

The purpose of PDS is to admit complementary specialists. A skill that is excellent on the safety-heavy ray but slightly slow on the speed-heavy ray would be rejected by a broad cone under CDS, but admitted under budgeted PDS. The paper's two-room gridworld examples show exactly this case, where a cautious detour option fails CDS on a wide cone but passes PDS with a small speed downside inside the budget.

1.4 The MDN: Learning the Motive Geometry Itself

The third pillar of SubRep is the Motive Decomposition Network, or MDN. In the bridge paper, the goal vector was hand-structured by a designer. SubRep makes the motive structure itself learnable. The MDN takes context, emits sub-aspect activations z, and mixes them into motive weights:

$$w = Az$$

This mixing is what induces the cone Wfor CDS or the cover $W_{ref}$for PDS.

The most important mechanical detail is that the MDN co-trains from the same residual stream that trains the option models. Every time an option fails or passes an admission test, the failure or success happens at specific directions in weight space, and those directions become training signals telling the MDN which sub-motives actually matter. Disentanglement regularizers, including orthogonality, sparsity, total correlation penalties, and anchor priors, keep the learned sub-aspects interpretable and stable over time.

There is also a clean monotonicity guarantee attached to this learning. As the MDN tightens the cone or refines the cover, all previously admitted options remain admitted, their margins weakly increase, and the cost of computing certificates goes down. So learning the motive geometry never destroys the library that was built under the older, looser geometry. This single property is what makes the co-learning loop safe to run continuously.

1.5 Positioning Summary of the Four Papers

Paper	Layer in the Stack	Core Idea	Mathematics	Status
OpenPsi (2013)	Appraisal and feeling	Emotion emerges from demand dynamics; six modulators; six mind agents on a shared AtomSpace	Fuzzy functions and nonlinear dynamics, no formal guarantees	Implemented and tested in three Minecraft-style scenarios
MetaMo theory (2025)	Abstract design language for the whole motivation system	Pseudo-bimonad of appraisal comonad and decision monad, lax distributive law, boundary contraction, tubular topology, five principles	Category theory plus analysis, with conditional stability guarantees	Pure theory
Bridge paper (2025)	Operationalizing MetaMo inside Hyperon	OpenPsi as appraisal, MAGUS as decision, individuation and transcendence overgoals, explicit update equations, PLN inference control	Concrete equations inheriting MetaMo's guarantees	Theory plus hypothetical walkthroughs, no experiments
SubRep (Sept 2025)	From motives to certified, composable skills	CDS/PDS admission gates, learnable MDN motive geometry, predictive coding execution, cross-paradigm composition, EASA, Waveformer/MORK sketch, conservative attention	Residuated quantale, with the densest guarantee set of the four	Theory plus tiny two-room case studies, experiments explicitly named as the next major step

2. Motivational State: From Demands to $G \times M$to a Motive Slice

This section compares how OpenPsi, MetaMo, the bridge paper, and SubRep represent motivational state. All of them are concerned with motivation, but they describe it in different ways. OpenPsi is the most biological because it starts from internal demands. MetaMo makes the motivational state more formal by using goals and modulators. The bridge paper connects OpenPsi-style demands with explicit goal structures. SubRep then changes the idea further by treating motivation as a region of possible motive states instead of one fixed state.

In OpenPsi, the agent carries internal demands such as energy, integrity, certainty, competence, and affiliation. These demands represent the basic needs of the agent. Along with these demands, OpenPsi also uses modulator values that influence the agent's mood and behavior. However, OpenPsi does not directly use explicit goal weights. A goal appears only when one demand moves too far away from its target level and becomes strong enough to guide action. So, in OpenPsi, motivation exists indirectly through the gap between current demand levels and their thresholds.

MetaMo formalizes this idea using:

$$X = G \times M$$

Here, Grepresents explicit goal intensities, and Mrepresents explicit modulators. This is an important change because goals become direct numerical values that can be updated using equations. They are no longer only side effects of demand homeostasis.

The bridge paper explains the internal structure of G. It places two overgoals first, followed by primary goals and anti-goals. The modulator vector Mcontains the six OpenPsi-style modulators: valence, arousal, approach, resolution, threshold, and securing. Because of this, the bridge paper acts like a bridge between OpenPsi's biological demand system and MetaMo's formal goal-modulator system.

SubRep changes the geometry of the problem. In MetaMo and the bridge paper, motivational state can be understood as one current configuration of $G$and M. But in SubRep, motivation is not a single point. It is a region of possible motive weight vectors. This region may be represented as a cone W, or as a finite reference set W_{ref}. Each weight vector $w$evaluates states using:

$$v_w(x) = w^T x$$

This means that every possible motive weight vector gives its own evaluation of the state. In simple terms, MetaMo asks, "What is my motivational state right now?" SubRep asks, "What are the motivational states I might plausibly be in, and what should I choose under that uncertainty?"

This shift makes SubRep more robust. In the bridge paper, the score function directly uses current goal and modulator values. Because of this, a temporary increase in arousal or another

modulator can strongly change the decision. SubRep avoids this by evaluating an option against every plausible motive state in W. Its worst-case evaluation prevents temporary mood changes from easily controlling the final choice.

One important point is that SubRep's motive slice does not directly contain modulators. There is no separate Mvector and no explicit variables such as valence or arousal. A similar role is partly handled by precision weighting in the predictive coding implementation. Precision decides which prediction errors become more important, which is functionally close to how modulators work in OpenPsi. However, at the formal level, SubRep removes the earlier "feel and choose" structure and replaces it with a "motives and admission" structure.

Overall, OpenPsi begins with biological demands and modulators. MetaMo turns this into the formal structure $G \times M$. The bridge paper adds overgoals, primary goals, anti-goals, and modulators into a clearer motivational architecture. SubRep then shifts from one motivational state to a region of possible motive states, making decisions more stable under motivational uncertainty.

2.2 Core Operators: Pseudo-Bimonad versus Residuated Quantale

This section compares the core operator structures used in MetaMo and SubRep. Both papers use algebraic ideas, but they use them for different purposes. MetaMo uses its operator structure to describe the interaction between appraisal and decision-making inside one agent. SubRep uses its operator structure directly as the planner's computational system.

In MetaMo, the main operator structure is:

$$F = D \circ \Psi$$

Here, Ψrepresents the appraisal comonad, and Drepresents the decision monad. Appraisal means how the agent evaluates or "feels" about a situation. Decision-making means how the agent chooses an action. So, MetaMo connects the appraisal side and decision side into one internal update cycle.

A key idea in MetaMo is the lax distributive law. This law makes sure that "feel first, then choose" and "choose first, then feel" do not produce completely unrelated results. They do not have to be exactly the same, but their difference must stay within a bounded distortion. This shows that MetaMo is mainly concerned with keeping the emotional and decision-making update cycle controlled.

SubRep's operator story is different because it is built around planning. In SubRep, every option $o$becomes a decision transformer:

$$T_o v = \hat{r}(x, o) + v\big(\hat{n}(x, o)\big)$$

Here, $\hat{r}(x, o)$is the predicted reward of taking option $o$in state x, and $\hat{n}(x, o)$is the predicted next state. This means the transformer shows how an option changes the value of future planning.

The set of these decision transformers forms a unital quantale. In this quantale, composition means sequencing. For example:

$$T_2 \otimes T_1$$

This means doing one option first and another option after it. The join operation means choice:

$$\vee$$

This represents choosing the option that gives the best backed-up value at the current state. The order relation is based on pointwise comparison of backed-up values.

The whole behavior of the SubRep planner is the join over the admitted option library:

$$B_B = \bigvee_{b \in B} T_b$$

This means the planner combines all admitted options in the library $B$and chooses the best value-supported behavior.

The interesting comparison is that both MetaMo and SubRep use quantales, but they use them for opposite jobs. MetaMo uses a commutative quantale like a measuring device. It measures how much distortion is allowed between two operation orders. SubRep uses a residuated quantale as the actual computational substrate of planning. In SubRep, the algebra is not just measuring the planner; it is the structure the planner runs on.

SubRep also defines a weakness functional:

$$w: K \to Q$$

This maps decision transformers into a commutative quantale. It follows the lax monoidality condition:

$$w(T_1 \otimes T_2) \leq w(T_1) \oplus w(T_2)$$

This means that the weakness of a composed plan is bounded by the combined weakness of its parts. In this way, MetaMo's idea that "things compose up to bounded slack" still survives in SubRep. However, it moves from the appraisal-decision interface to the weakness measure on plans.

The residuation part is the major new contribution of SubRep. Since the quantale is residuated, the system can solve for missing parts of a plan. If the desired specification is $S$and the system already has a suffix B, the weakest prefix that completes the job is:

$$A^* = S/B$$

Similarly, if the prefix $A$is already known, the weakest suffix can be written as:

$$B^* = A\backslash S$$

This is something the earlier papers could not do. OpenPsi could not even pose the question of missing behavior. The bridge paper's MAGUS system could score candidate actions, but it did not synthesize a missing action algebraically. SubRep can at least express this process formally.

This also connects to weakness-driven learning. Among all options that are consistent with the data and constraints, SubRep chooses the least committed option:

$$T^* = \bigvee \mathrm{Feas}(D, W)$$

This means the system avoids overcommitting too early. It selects the weakest sufficient option that still satisfies the requirements.

Overall, MetaMo's operator system is centered on internal appraisal and decision updates. SubRep's operator system is centered on planning, option composition, and plan completion. MetaMo uses quantales to measure bounded distortion, while SubRep uses a residuated quantale as the actual algebra of planning.

2.3 Overgoals versus the MDN: Who Decides the Structure of Motivation

This is one of the strongest differences between the bridge paper and SubRep. Both papers discuss how motivation can be structured, but they answer the question differently. The bridge paper gives a top-down architectural answer, while SubRep gives a bottom-up learning-based answer.

In the bridge paper, the structure of motivation is mostly fixed by the designer. The designer defines two main overgoals: individuation and transcendence. The designer also defines primary goals such as help, curiosity, novelty, self, ethics, and social behavior. These are then connected to the modulator system. Individuation increases threshold and securing, while transcendence increases arousal and approach.

The decision update also includes the tension between these two overgoals:

$$-\lambda_{\mathrm{Ind}}\, g_{\mathrm{over}}^{\mathrm{Ind}} + \lambda_{\mathrm{Trans}}\, g_{\mathrm{over}}^{\mathrm{Trans}}$$

This term shows individuation and transcendence pulling in opposite directions. Individuation acts like a brake because it supports safety, caution, and stability. Transcendence acts like an accelerator because it supports exploration, novelty, and growth. This is an elegant structure, but it is also hand-designed. The overgoals, primary goals, and their connections are all manually placed into the system.

SubRep gives a different answer. In SubRep, the designer still seeds a few top-level motives with default weights, but the detailed decomposition into sub-motives is learned by the Motive Decomposition Network, or MDN. The MDN learns from admission residuals. When a candidate option fails a CDS test, the failure happens in specific directions of motive weight space. These failure directions become learning signals for the MDN.

Over time, the matrix Ain the motive decomposition equation reorganizes itself:

$$w = Az$$

Here, the latent coordinates $z$gradually begin to represent the actual trade-offs that the planner keeps certifying. In simple words, the MDN learns which motivational axes matter by observing where options pass or fail.

The examples in SubRep make this clearer. In the robot example, compassion splits into sub-motives such as proxemics, yielding, jerk limits, and risk margin. In the chatbot example, compassion splits into empathic tone, harm guardrails, privacy, and fairness. These splits are not directly specified by the designer. They emerge because these are the dimensions that repeatedly matter during planning and admission testing.

A strong connection appears in SubRep's treatment of the metagoal called "moderated pace of goal self-modification." This is related to Goertzel's fixed-point goal-stability idea and also to MetaMo's incremental objective embodiment principle. In MetaMo, goal stability is mainly added as a principle or axiom. In SubRep, it becomes a motive that the MDN can decompose and learn from experience.

The MDN can learn sub-aspects of moderation such as goal drift speed, sudden bursts of goal change, evidence required for change, rollback availability, explanation burden, distance from ethical boundaries, and different timescale gates. These gates allow fast changes to small heuristics while slowing down changes to top-level objectives. This means a stability constraint that MetaMo adds from above can become, in SubRep, a learned motivational structure.

There is also an important comparison between individuation/transcendence and CDS/PDS. In the bridge paper, individuation and transcendence modulate intensity. They affect how cautious, exploratory, or self-modifying the agent becomes. In SubRep, CDS and PDS are not modulators. They are admission gates that decide whether a skill enters the library.

CDS is close to the individuation instinct because it uses worst-case evaluation. An option is admitted only if it is safe or useful under all plausible motive weightings. PDS is closer to the transcendence instinct because it allows specialist options that help some directions and mildly hurt others, as long as the downside stays within the allowed budget ρ.

So, the individuation/transcendence duality does not disappear in SubRep. It appears again as the difference between two admission rules. CDS is conservative and safety-focused, while PDS is more open to complementary specialization.

Overall, the bridge paper asks the designer to decide the structure of motivation in advance. SubRep allows the agent to gradually discover the structure of motivation through the MDN and admission residuals. This makes SubRep more adaptive because sub-motives are learned from the trade-offs the planner repeatedly faces.

2.4 Stability: Contraction Band versus Monotone Certificates

This section compares how MetaMo and SubRep handle stability. MetaMo's stability story is mainly dynamical, while SubRep's stability story is more order-theoretic.

In MetaMo, there is a safe region $R$and a boundary band B_η. When the agent's state moves close to the boundary, the update function Fcontracts distances near the edge:

$$d(F(x), F(y)) \leq c \cdot d(x, y) + \varepsilon, c < 1$$

This means that if the agent's state starts moving toward an unsafe or unstable region, the update pulls it back toward stability. The bridge paper makes this more concrete by defining Ras the set of states where individuation remains above a safety floor and the goal vector norm stays bounded.

Self-continuity in the bridge paper comes from the Lipschitz self-model map $H$and the blending update:

$$x_{t+1} = (1 - \alpha)\, x_t + \alpha\, x^*$$

Here, $\alpha$is controlled by the two overgoals. This means the agent's self-model does not change too suddenly. Instead, updates are blended gradually, which helps preserve continuity.

SubRep handles stability differently. Its stability is based on certificates rather than contraction of state trajectories. One key guarantee is join-safety. This means that combining admitted options through joins and sequencing preserves the improvement certificate. So, as the admitted library grows, previous certifications are not broken.

Another guarantee is monotone persistence. When the MDN tightens Wor refines W_{ref}, everything previously admitted stays admitted, and margins weakly increase. This means learning and refinement do not destroy the earlier admitted structure.

SubRep also uses a two-timescale convergence argument. The motive geometry changes slowly, while option learning happens faster inside that geometry. Because of this separation of speeds, the option library can settle into a stable admitted structure over time. This is the closest connection to MetaMo's contraction idea. MetaMo stabilizes the state by pulling it back

from the boundary. SubRep stabilizes the planner by making motive learning slow and option learning fast.

SubRep also handles a problem that MetaMo does not directly address: library bloat. As an agent learns more skills, the planner could become too large and inefficient. SubRep avoids this through logarithmic library growth under mild separability conditions and through the preference for the weakest sufficient option. These are anti-bloat mechanisms because they prevent the planner from drowning in too many accumulated skills.

However, SubRep is weaker than MetaMo in one area: self-continuity. MetaMo has a self-model map $H$and a Lipschitz-style bound that controls how fast the agent's understanding of itself can change:

$$d_M(H(x), H(y)) \leq L_H \, d_X(x, y)$$

SubRep does not have the same direct self-continuity guarantee. The closest idea is the moderation metagoal, which controls the pace of goal change. But this is not the same as proving that the agent remains recognizable to itself while learning new skills.

Therefore, a fair criticism is that SubRep certifies that skills help and that the option library remains stable, but it does not fully certify that the agent's self-representation remains continuous. MetaMo is stronger in self-continuity, while SubRep is stronger in skill admission, library growth control, and planner stability.

3. Execution Engines and Cross-Paradigm Learning: Mind Agents versus Predictive Coding and EASA

3.1 What We Are Comparing in This Part

This part compares how each framework actually runs in practice. OpenPsi runs through six mind agents working on a shared AtomSpace, while SubRep uses predictive coding networks and admission-based planning. The comparison also includes how OpenPsi modulators relate to SubRep precisions, how hand-coded emotion dynamics differ from learned motive geometry, and how SubRep adds a new cross-paradigm learning system where neural, logical, and evolutionary methods can all connect through one common interface. This larger cross-paradigm system is formalized in SubRep as EASA, the Evo-Adhesive Smooth Atlas.

3.2 OpenPsi's Mind Agents versus SubRep's Predictive Coding Stack

OpenPsi's execution model is based on architectural emergence. It uses six small mind agents: PerceptionUpdater, MonitorChanges, DemandUpdater, ModulatorUpdater, FeelingUpdater, and ActionSelection. These agents run at the same time and do not directly communicate with each other. Instead, they coordinate by reading and writing Atoms in a shared knowledge base called the AtomSpace.

This design matches the PSI-theoretic idea that emotion and behavior should emerge from the interaction of simple processes. There is no single central controller or executive system. Instead, the complete emotional and motivational behavior appears from the combined activity of the six agents. This makes OpenPsi biologically inspired and elegant at the architectural level.

However, the limitation is that OpenPsi does not learn its own structure. Many of its update rules are fixed in advance. For example, the fuzzy equality functions, activation and resolution equations, and emotion intensity tables are hand-coded. The main adaptive part is the drifting demand target range, which gives the agent something like a slowly developing personality. But the architecture itself does not reorganize or learn new motivational dimensions.

SubRep replaces this with a predictive coding stack. In SubRep, the agent maintains a hierarchical generative model. Each layer predicts the layer below it, and the difference between the actual value and predicted value becomes the prediction error:

$$\varepsilon_\ell = x_\ell - \hat{x}_\ell$$

These errors flow through local error nodes, and synapses update using local Hebbian-style rules:

$$\Delta W_\ell \propto \varepsilon_\ell \, x_{\ell-1}^T$$

This is important because SubRep does not require normal global backpropagation. Learning happens through local prediction errors, which makes the system more biologically plausible and also better suited for continual adaptation.

SubRep builds on two active predictive coding architectures. The first is Rao's APC, which is hierarchical and uses hypernetworks. In that model, higher levels generate the parameters of lower-level dynamics and policy modules. It also separates micro steps for execution from macro steps for abstraction. The second is Ororbia's ActPC, which is fully backprop-free and uses two coupled predictive coding circuits: a world model and an actor. These are balanced by instrumental and epistemic signals.

From these predictive coding systems, SubRep extracts a few common plug points. These include a reward head, a successor-feature head, candidate options, and precision controls:

$$\hat{r}(x, o)$$
$$\hat{n}(x, o)$$

The reward head estimates the value of taking an option, while the successor-feature head estimates the next state or future features after taking that option.

The reason predictive coding is useful for SubRep is that the two important admission operations, CDS and PDS, fit naturally into predictive coding. The planner needs to compare backed-up values across options, and predictive coding networks already perform soft competition through their internal dynamics. Similarly, the robust infimum over the motive cone can be computed either in closed form or by evaluating a small number of extreme rays suggested by the MDN.

Admission margins and Pareto gaps can also be attached directly to prediction error nodes as residuals. This means the same error signal can train the world model, the option heads, and the MDN at the same time. This is a major advantage over ordinary continual backpropagation, especially when motives are changing over time. Predictive coding gives more factorized and local credit assignment, so the system can identify which specific factor went wrong instead of spreading blame through one global gradient.

A key comparison is that SubRep's precisions play a role similar to OpenPsi's modulators. In OpenPsi, resolution affects how carefully the agent perceives, securing threshold affects how often it checks for change, and selection threshold affects how easily it changes motives. In SubRep, precisions on error nodes control which prediction errors are amplified and which are reduced. For example, early in learning, noisy novelty signals can be downweighted so that safety-related errors dominate the formation of the MDN.

So, both modulators and precisions perform context-sensitive gain control over cognition. However, there are two major differences. First, SubRep's precisions are learned and can be specific to individual channels, while OpenPsi's modulators are more global and updated by fixed formulas. Second, OpenPsi interprets modulator configurations as emotions, while

SubRep does not treat precision values as emotions. SubRep keeps the gain-control mechanism but drops the emotional interpretation.

This creates an important trade-off. OpenPsi's emotion tables are readable and easy to interpret, but they are fragile because the prose explanation and the mathematical formula can sometimes disagree. For example, in the OpenPsi documentation, there is a mismatch around the selection threshold and competence relationship. SubRep avoids this kind of direct prose/formula mismatch because it does not use readable emotion tables in the same way. However, learned precisions are less transparent. In simple terms, OpenPsi is more interpretable but more hand-coded, while SubRep is more adaptive but more opaque.

3.3 The Modality-Neutral Interface: Logic and Evolution as First-Class Citizens

The earlier papers mention the importance of integrating reasoning, but they do not fully make reasoning equal to other learning methods. The bridge paper's strongest contribution in this area is using motivation to guide PLN inference. It treats each candidate inference rule like a stimulus to be appraised, scores it using the usual goal and overgoal formula, and then runs only the top selected inferences. This is useful, but logic is still being controlled by motivation from the outside. Logic is not yet participating in motivation and planning on equal terms.

SubRep goes further by defining a minimal common interface. Any generator that can export an expectation-model summary can become a decision transformer and be screened by the same CDS/PDS gates as a neural option. The summary mainly consists of:

$$(\hat{r}, \hat{n})$$

This means that a method only needs to provide an estimated reward and an estimated next-state or successor-feature effect. Once it does this, it can be treated like any other option by the planner.

SubRep applies this to non-neural methods also. For probabilistic logic with backward chaining, a macro-step of rule application becomes a transformer. The system may unify a goal with a rule head, replace it with instantiated antecedents, and continue until the subgoal frontier closes or a budget limit is reached. This logical option can be represented as:

$$T_{o_r}$$

Here, the reward estimate is based on accrued utility during the proof process, while PLN truth values help estimate the reliability of the result. When data are limited, abstract interpretation and type or shape analyses can provide bounds.

SubRep also treats evolutionary program learning in the same way. Evolved programs export the same summaries and are screened by the same CDS/PDS gates. This means neural, logical, and evolutionary methods are no longer separate bolt-ons. They become different suppliers of options to the same planner.

The two-room examples in the paper support this idea. In one case, explicit logical inference finds the correct goal decomposition quickly, while gradient search struggles. In another case, logic, evolution, and gradients together perform better than any pair of two methods. Even though these examples are small, they show the main point clearly: different paradigms are good at different parts of the problem. A shared admission gate allows them to be combined without making the system chaotic.

So, the main contribution of SubRep here is the modality-neutral planner interface. Instead of saying that neural networks, logic, and evolution are separate modules, SubRep makes all of them produce the same type of planning object. After that, the same algebra and the same admission rules decide what gets used.

3.4 EASA: Gluing the Three Paradigms with Guarantees

EASA, or the Evo-Adhesive Smooth Atlas, is SubRep's formal way of explaining how neural, logical, and evolutionary methods fit together. The name is based on the idea of an atlas in geometry. Just like an atlas covers a manifold using overlapping charts, EASA covers the goal region using overlapping local models.

The smooth charts are neural patches. These are local models, written as C_i. Each chart is contractive in the planning metric, with Lipschitz constant:

$$\mathrm{Lip}(C_i) \leq \lambda < 1$$

Each chart also approximates the true transformer within a small error on its own patch:

$$d_M(T \mid_{U_i}, C_i) \leq \frac{\varepsilon}{6}$$

The logical part performs the gluing. Guard predicates $G_i$decide which chart applies in which region. The overlaps between charts are thin, and charts must be consistent where they meet. On the overlap, they can disagree only within a bounded error:

$$\mid C_i(x) - C_j(x) \mid \leq \frac{\varepsilon}{6}$$

The evolutionary part searches over possible amalgams, meaning combinations of glued charts. A bounded-interaction condition keeps the error of an amalgam with $m$charts under:

$$\Delta \cdot m$$

This gives a clear division of roles. Neural models provide smooth local approximations. Logic decides where each model applies and how they connect. Evolution searches over combinations of these pieces.

There is also a strong connection between EASA and MetaMo. Contractive charts are similar to MetaMo's contraction law, but applied locally instead of globally. Logical gluing with bounded seam disagreement is also similar to MetaMo's lax coherence idea. In MetaMo, exact equality between operation orders is replaced by bounded distortion. In EASA, exact matching between behavior charts is replaced by bounded disagreement on overlaps.

The main theorem-level idea is that the glued object preserves CDS/PDS certificates on the same motive slice. This means that when behaviors are composed from charts, the safety or improvement guarantees are not silently lost. This is important because one danger of combining paradigms is that the final combined system may break the guarantees of the individual pieces. EASA tries to prevent that.

The paper is also careful about the limits of compositional weakness. It does not claim that perfect multiplicative weakness bounds are always possible. Instead, what survives is a sublinear-penalty bound. This means weakness can still be controlled, but not in an ideal exact way. The paper treats compositional weakness as a design principle and soft constraint rather than a perfect hard guarantee. This makes the framework more realistic.

There is also a categorical interpretation in the later part of the paper. Options can be represented as polynomial functors with a lens structure. The forward map executes the option, while the backward map propagates value updates. The paper also defines different weakness functionals, such as interface complexity, maximum branching, entropy-weighted weakness, and Kolmogorov-style weakness. These can be used as regularizers.

For this comparison report, the simple takeaway is enough: EASA gives SubRep a formal way to combine neural, logical, and evolutionary learning. It does not just place them side by side. It gives them a shared planning interface, a gluing mechanism, and bounded guarantees. This makes SubRep's execution engine more flexible than the earlier systems because it can use multiple learning paradigms under one admission and planning structure.

4. Hyperon Integration, Attention Economics, Guarantees Side by Side, and Honest Limitations

4.1 What We Are Comparing in This Part

This final part compares where each framework is implemented, how each framework deals with limited cognitive resources, what kind of guarantees each paper provides, and what limitations should be honestly mentioned. The comparison includes OpenPsi's AtomSpace implementation, the bridge paper's Hyperon direction, and SubRep's newer integration with MORK and Waveformer-style architectures. It also compares attention economics, especially ECAN/RFS-style resource allocation versus SubRep's conservative attention mechanism. Finally, this part gives a side-by-side view of the formal guarantees and a maturity assessment of all four papers.

4.2 Substrate: AtomSpace Then, MORK and Waveformers Now

OpenPsi was implemented on the original OpenCog AtomSpace. Its design used a client-server style structure where agents written in different languages communicated through a common message protocol. Redis was used underneath as part of the supporting infrastructure. In this setup, the six OpenPsi agents worked through the shared AtomSpace instead of communicating directly with each other. This made AtomSpace the central shared substrate for perception, demand updating, modulator updating, feeling formation, and action selection.

The bridge paper moves the discussion toward Hyperon. Instead of the original AtomSpace, it uses the newer metagraph AtomSpace idea. It also places PLN as the reasoning engine and PRIMUS as the broader cognitive architecture. In this version, the motivation system does not only select external actions. It also steers inference by influencing which reasoning steps should receive more attention.

SubRep targets an even newer layer of this stack. Its implementation sketch places the full system inside a Waveformer-style transformer. In this setup, the MDN and option heads are placed as heads on a shared trunk. CDS and PDS act as gates inside the network, while backed-up value computations and robust reductions are fused into batched matrix operations.

For PDS, the paper describes stacking the reference rays of W_{ref}into a matrix so that all rays can be evaluated efficiently:

$$W_{\text{ref}} \in \mathbb{R}^{d \times K}$$

This allows one large matrix multiplication to evaluate all reference rays across all candidate options. So, SubRep is not only a theoretical planner. It is also designed with scalable implementation in mind.

SubRep also maps naturally onto MORK, which is described as a multiresolution DAG runtime under a Hyperon AtomSpace implementation. This is important because MORK supports the

two different timescales that SubRep needs. The fast micro loop can run predictive coding inference and CDS/PDS gating, while the slower macro loop can handle MDN updates, evolutionary search, and guard compilation.

Certified libraries are maintained per shard, and double-buffered staging is used so that admissions can change atomically at shard boundaries. Only small typed messages, such as indices, margins, and statistics, need to cross shards. This makes the system more practical for distributed or large-scale execution.

One honest point is that this may look like premature optimization because SubRep does not yet have experiments, even for a smaller version. However, the paper's defense is that AGI-scale systems need implementation strategy to be considered early. The MVP ladder at the end also makes the proposal more realistic. It starts with a simple MVP-0 using a box cone and conservative attention, then moves to robust PDS, then two-level motives, and finally the full tri-modal EASA-Waveformer version. This shows that the system is not proposed as one huge monolith, but as something that could be built step by step.

4.3 Attention Economics: Conservative Attention versus RFS and ECAN

Admission decides which skills or options are allowed into the library, but it does not decide what should run right now under a limited compute budget. This is where attention economics becomes important.

In PRIMUS, the existing answer comes from ECAN-style attention currency and the RFS mechanism. In RFS, goals can issue non-conserved promissory notes to potential subgoals. Later, subgoals can redeem these promises for actual attention currency. Goals can also withdraw promises, and joint requirements can use constraint-based notes. This creates a detailed economic system for deciding where cognitive resources should go.

SubRep argues that once CDS/PDS certificates already exist, much of this extra machinery becomes unnecessary. The reason is that CDS/PDS already compute robust margins and admission evidence. So, instead of using a complicated promise-and-redemption system, SubRep proposes a simpler conserved attention pool.

The attention allocation is based on a softmax over the margins produced by the gates:

$$a(o) = a_{\text{prev}}(o) \cdot \gamma + A_{\text{free}} \cdot \frac{\exp\left(m_o(x; W)/\tau\right)}{\sum_{o'} \exp\left(m_{o'}(x; W)/\tau\right)}$$

Here, $a(o)$is the attention allocated to option o. The previous attention $a_{\text{prev}}(o)$is decayed by γ. The remaining free attention $A_{\text{free}}$is distributed using a softmax over the margin $m_o(x; W)$. The temperature τcontrols how sharp or smooth the allocation is.

This design has a few advantages. First, conservation is automatic because the softmax normalization distributes a fixed pool. Second, allocation is deterministic and does not depend

on later redemption. Third, the decay rate $\gamma$and temperature τstill provide adaptive dynamics, similar to what RFS got from withdrawal and redemption timing.

The paper does not completely reject the economic insight of ECAN and RFS. It keeps the Baum-style idea that no cognitive process should receive reward or credit for free. Cognitive reward should remain bounded by allocated attention. The difference is that SubRep simplifies the mechanism because the admission gates have already done much of the epistemic work.

Across the four-paper comparison, this gives a clear progression. OpenPsi has almost no explicit resource economics because the six agents simply run. PRIMUS adds a sophisticated attention economy through ECAN and RFS. SubRep then simplifies the economy by using margins already produced by CDS/PDS. So, SubRep does not remove attention economics; it makes it more directly connected to certified planning.

4.4 Guarantees Side by Side

The four frameworks differ strongly in what they can actually prove. OpenPsi is the most empirical, MetaMo is more theoretical, the bridge paper operationalizes MetaMo, and SubRep has the densest guarantee set.

Framework	Evidence	Summary
OpenPsi	Empirical only	OpenPsi does not give formal proofs. Its evidence comes from experiments in three scenarios using one fixed parameter set. It shows emotions matching situations, Lewis's trigger, self-amplification, and self-stabilization phases in plots. Importantly, OpenPsi is the only one among the four that was actually run experimentally.
MetaMo	Conditional stability guarantees	MetaMo proves stability properties if its assumptions hold. Boundary contraction keeps trajectories inside a safe region. The Lipschitz self-model bound keeps self-drift finite. Tubular path-connectedness plus local contraction supports incremental convergence to reachable targets. The lax distributive law bounds distortion between different operation orders.
Bridge Paper	Instantiated MetaMo guarantees	The bridge paper mostly inherits MetaMo's guarantees and makes them more concrete. It adds explicit update equations, a safe region based on individuation and goal norm, and an adaptive blending rate. Its contribution is more operational than theorem-heavy.
SubRep	Dense formal guarantee set	SubRep provides join-safety, monotone persistence, two-timescale convergence, sample complexity bounds, robustness under bounded model error, logarithmic library growth, budget conservation, non-increasing regret surrogate for attention, risk-sensitive extensions, and EASA sufficiency and approximation theorems. It also includes a negative result about compositional weakness bounds.

For MetaMo, the main stability idea is that the update function contracts near the boundary of the safe region:

$$d(F(x), F(y)) \leq c \cdot d(x, y) + \varepsilon, c < 1$$

For SubRep, one important robustness correction is based on bounded model error:

$$| \hat{r} - r | \leq \varepsilon_r$$
$$\| \hat{n} - n \|_\infty \leq \varepsilon_n$$

This means that if the reward model and successor-feature model are wrong only within known bounds, the admission certificates can be made conservative enough to remain valid.

SubRep also extends its guarantees to risk-sensitive objectives such as entropic risk and CVaR. In one case study, increasing risk aversion changes which detour the planner prefers. This shows that SubRep's formalism is not limited only to expected reward; it can also handle different attitudes toward risk.

A very important honest point is that OpenPsi remains the only system in the set with real empirical evidence. The later papers are stronger mathematically, but they have not been tested in the same way. So, the comparison is not simply "later is better." OpenPsi is weaker formally but stronger experimentally. SubRep is stronger formally but still needs implementation and empirical validation.

---

5. Formula Evolution Across the Four Papers: What Changed, Why It Changed, and What Every Term Means

Evolution 1: How Satisfaction and Demands Became a Motive Slice

OpenPsi's formula (where it starts)

OpenPsi measures how satisfied each demand is using a piecewise rule built on one tiny building block:

$$\text{fuzzy\_equal}(x, t, a) = \frac{1}{1 + a(x - t)^2}$$

What each term compares:

- $x$is the current level of a demand, for example the current energy level
- $t$is the target level the agent wants that demand to sit at
- $a$controls how sharply satisfaction drops when you move away from the target (they use $a = 150$for demands, which is a fairly sharp bell)
- The output is 1 when $x$exactly hits the target and falls toward 0 as $x$drifts away

So motivation in OpenPsi is literally just "how far is each need from its healthy level." There is no goal vector anywhere. The problem: everything is hand-tuned. Why 150? Why this bell shape? Nobody can prove anything about an agent built from these curves.

MetaMo's formula (the formalization)

MetaMo throws away the fuzzy curves and says the agent's whole motivational state is one object:

$$X = G \times M$$

- $G$is a vector of goal intensities, how strongly the agent wants each goal right now
- $M$is the vector of modulators, the mood dials like valence and arousal

Goals become explicit numbers you can write update equations for, instead of side effects of demand gaps.

SubRep's formula (the final shift, and why)

SubRep replaces the single state with a scoring function over a region of weight vectors:

$$v_w(x) = w^T x$$

- $x$is the feature vector of the current state

- $w$is one possible motive weighting, one way of caring about the features
- $v_w(x)$is the value of that state under that particular weighting
- $w$is never a single fixed vector, it lives in a cone Wor a finite cover W_{ref}

Why they reset to this form: the team realized that any formula using the current $G$and Mdirectly inherits their noise. If arousal spikes for one cycle, the decision flips. A 155-page framework about robust goal pursuit cannot sit on top of a mood reading. So they made the formula deliberately uncertain about its own motives, and pushed the question to "what holds across all plausible w" instead of "what does the current $w$say."

Evolution 2: From Appraisal Updates to No Appraisal at All

OpenPsi and the bridge paper

The bridge paper writes appraisal as:

$$\Psi((G, M), s) = (G, M')$$

- $(G, M)$is the state before the stimulus
- $s$is the incoming stimulus, a new paper, a user query, an event
- $M'$is the updated mood, the goals Gstay untouched

The whole meaning is in what does not change: a stimulus changes your mood, never your goals directly.

SubRep

This formula has no successor in SubRep. There is no Ψ, no M, no mood update. The closest functional replacement is the precision weighting inside the predictive coding networks, where precisions decide which prediction errors get amplified. The team did not "fix" the appraisal formula, they delegated it upward. Section 4.1 of SubRep states that MetaMo answers "what matters now" and hands down the motive set, and SubRep starts from there. So one formula in the chain was retired rather than evolved, which is itself worth flagging in a defense.

Evolution 3: The Decision Formula, the Big One

This is the chain the 155-page paper exists to finish, so go slow here.

Step 1: OpenPsi's selection (a coin flip)

```
IF random(0,1) < SELECTION_THRESHOLD
   select the demand with lowest satisfaction
ELSE
```

randomly pick a demand

- With probability equal to the selection threshold, chase the most urgent need
- Otherwise pick randomly, deliberate noise so the agent does not get stuck

It works for a Minecraft robot, but there is no notion of an action being good across multiple objectives. One demand wins, that is the whole decision.

Step 2: The bridge paper's score (a weighted sum with a brake and accelerator)

$$\text{score}(a) = \sum_i w_i \; g_i \, m_i \, \text{corr}(a, g_i) \; - \; \lambda_{\text{Ind}} \, g_{\text{over}}^{\text{Ind}} \; + \; \lambda_{\text{Trans}} \, g_{\text{over}}^{\text{Trans}}$$

What each term compares:

- $w_i$is the fixed designer weight of goal i
- $g_i$is how intense goal $i$is right now
- $m_i$is the relevant modulator value, so mood directly scales the score
- $\text{corr}(a, g_i)$measures how well action $a$lines up with goal i
- $-\lambda_{\text{Ind}} \, g_{\text{over}}^{\text{Ind}}$is the individuation brake, high self-preservation drive pulls every score down
- $+\lambda_{\text{Trans}} \, g_{\text{over}}^{\text{Trans}}$is the transcendence accelerator, high growth drive pushes scores up

This is much richer than the coin flip, but notice the weakness: the score is evaluated at one point. One set of g_i, one set of m_i, right now. A temporary mood swing changes the ranking. Also, the score ranks actions for this moment but says nothing about whether a skill is worth keeping forever.

Step 3: SubRep's reset, the worst-case margin

First, every option becomes a decision transformer:

$$T_o v = \hat{r}(x, o) + v(\hat{n}(x, o))$$

- $\hat{r}(x, o)$is the predicted reward accumulated while running option $o$from state x
- $\hat{n}(x, o)$is the predicted state features after the option finishes
- $v$is the downstream value function, so the transformer converts future value into present value through the option

Then the baseline, the best the agent can already do with its current library B:

$$B_{\text{base}}(x; w) = \max_{b \in B} \left(\hat{r}(x, b) + w^T \hat{n}(x, b) \right)$$

And finally the formula the whole paper is built around, the CDS margin:

$$m_o(x) = \inf_{w \in W} (B_o(x; w) - B_{\text{base}}(x; w))$$

What each term compares:

- $B_o(x; w)$is the backed-up value of the new candidate option under weighting w
- $B_{\text{base}}(x; w)$is the backed-up value of the existing library under that same weighting
- The subtraction is the improvement the candidate offers under one weighting
- The inf over Wtakes the worst case across every plausible weighting in the cone
- Admit the option only if even the worst case is nonnegative

Why the team reset to this exact form: the bridge paper's score answers "what looks best to me right now," and Sutton's reward-respecting subtasks answer "what helps the one reward I was given." Both fail the same way, they are tuned to a single weighting. The infimum is the mathematical move that kills that failure. If $m_o(x) \geq 0$, the option helps no matter which way the agent's priorities lean inside the cone. The brake and accelerator from the bridge paper did not disappear, they got absorbed into the shape of Witself. A safety-tilted cone is the individuation brake; a wide exploratory cone is the transcendence accelerator.

Step 4: PDS, the relaxation they added when CDS proved too strict

The worst case can be too harsh. A specialist skill that is great for safety but slightly slow gets killed by the infimum. So they added the Pareto rule on a finite set of reference rays. The component gap:

$$\Delta_k(x, o) = B_o(x; w^{(k)}) - B_{\text{base}}(x; w^{(k)})$$

And the budgeted admission test:

$$| \{k : \Delta_k(x, o) \geq \varepsilon\} | \geq qK, \Delta_k(x, o) \geq -\rho \quad \forall k$$

- $w^{(k)}$is one of the Kreference weightings in W_{ref}
- $\varepsilon$is the bar for what counts as a real improvement on a ray
- $q$is the coverage fraction, on how many rays must the option clearly win
- $\rho$is the downside budget, the maximum loss tolerated on any single ray

So the final design has two formulas instead of one on purpose: CDS for safety-uniform skills, PDS for complementary specialists.

Evolution 4: Who Generates the Weights

The bridge paper hand-writes the goal vector. SubRep replaces the hand with one small equation:

$$w = Az$$

- $z$is the vector of sub-aspect activations the MDN computes from context
- $A$is the learned mixing matrix
- $w$is the resulting motive weighting that feeds the cone or the cover

The reason this formula exists: the CDS margin needs a cone W, and somebody has to say where the cone is. The 2013 answer would be "the designer." SubRep's answer is that the matrix Areorganizes from admission residuals, every failed test points at a direction in weight space that mattered, and that direction trains A. The motive geometry becomes a learned object instead of a config file.

Evolution 5: The Stability Formula

MetaMo's contraction:

$$d(F(x), F(y)) \leq c \cdot d(x, y) + \varepsilon, c < 1$$

- $d$is distance in motivational state space
- $F$is one full feel-and-choose update
- $c < 1$means two nearby states get pulled closer after the update, so trajectories near the boundary cannot escape
- $\varepsilon$is a small tolerated slack

The bridge paper's blending:

$$x_{t+1} = (1 - \alpha)\, x_t + \alpha\, x^*, \alpha = \alpha_0(1 - g_{\text{over}}^{\text{Ind}}) + \beta_0\, g_{\text{over}}^{\text{Trans}}$$

- Each cycle moves only a fraction αtoward the target x^*
- High individuation shrinks α, slow careful change; high transcendence grows it, faster growth
- (Keep the standing note: other sections of the bridge paper write $\alpha$with individuation in a denominator instead, a draft-stage inconsistency)

SubRep's replacement, the model-error cushion:

$$\inf_{w \in W} (\hat{r} + w^T \hat{n}) \;\geq\; \inf_{w \in W} (r + w^T n) \;-\; \varepsilon_r \;-\; \| W \|_{\max,1} \; \varepsilon_n$$

- $\varepsilon_r$bounds how wrong the reward model can be, $| \hat{r} - r | \leq \varepsilon_r$
- $\varepsilon_n$bounds the successor-feature error, $\| \hat{n} - n \|_\infty \leq \varepsilon_n$
- $\| W \|_{\max,1} = \sup_{w \in W} \| w \|_1$converts feature error into value error, a bigger cone amplifies model mistakes more
- The whole inequality says: subtract this cushion from the margin, and the certificate survives even if your models are wrong within known bounds

Why they reset the stability story: MetaMo stabilizes trajectories, SubRep stabilizes certificates. Instead of "the state cannot run away," the guarantee becomes "the admission decision cannot be invalidated, not by library growth (join-safety), not by tightening the cone (monotone persistence), not by bounded model error (the cushion above)." And the contraction idea did not vanish, it survives in two smaller places: the EASA charts with $\text{Lip}(C_i) \leq \lambda < 1$, and section 6.2 where the goal-drift-contraction metagoal becomes something the MDN learns sub-aspects of.

Evolution 6: The Attention Formula

PRIMUS RFS computed promises:

$$\text{RFS}(B \mid A) = \text{promise}(A) \cdot P(\text{PredictiveImplication } B \to A)$$

SubRep replaces the whole promise-and-redemption economy with one line:

$$a(o) = a_{\text{prev}}(o) \cdot \gamma + A_{\text{free}} \cdot \frac{\exp\,(m_o(x; W)/\tau)}{\sum_{o'} \exp\,(m_{o'}(x; W)/\tau)}$$

- $a_{\text{prev}}(o) \cdot \gamma$is yesterday's attention decayed, nothing accumulates forever
- $A_{\text{free}}$is the leftover budget being redistributed
- $m_o(x; W)$is the same CDS margin from Evolution 3, reused as the allocation signal
- $\tau$is the temperature, sharp winner-take-most when low, smooth sharing when high
- The softmax denominator forces conservation, the pool always sums to the budget

Why the reset: RFS allocated attention using PredictiveImplication strengths because nothing better existed. After CDS/PDS, something better does exist, the robust margins are already computed at admission time. So the team deleted the two-stage economy and let the certificate itself decide the budget. One number, the margin m_o, now drives both whether a skill exists and how much compute it gets.

6. Limitations Across All Four Papers

OpenPsi has several limitations. It uses a small fixed demand set, hand-tuned fuzzy formulas, and a tiny toy world. Its emotion-labeling layer is also more like an interpretive overlay than a deeply learned mechanism. Another issue is the prose/formula inconsistency around selection threshold and competence, which makes the model less clean. However, OpenPsi's biggest strength is that it was actually implemented and tested. This also highlights how much of the later theory has not yet touched reality.

MetaMo is much more theoretically ambitious, but it also has limitations. The authors themselves admit that its dynamical principles are most convincing for roughly human-like cognitive architectures. Some superintelligent systems might not have the kind of coherent self-model that MetaMo tries to preserve. MetaMo also does not provide a real learning mechanism. Its categories and structures are mainly descriptive, and the framework does not show how the agent adapts those structures over time.

The bridge paper has a different weakness. Its structure is still mostly hand-designed. The overgoals, primary goals, couplings, and λcoefficients are all manually chosen. It provides useful hypothetical walkthroughs, but not experiments. There is also a small notational issue in the formula for α, where different sections use slightly different forms, such as a denominator form versus a $\left(1-g_{\text{over}}^{\text{Ind}}\right)$multiplier form. This suggests the paper is still at a draft-stage level of maturity.

SubRep has the most advanced formal structure, but it also has the largest implementation gap. Many of its guarantees depend on assumptions such as good model error bounds, separability, stable two-timescale learning, and meaningful motive geometry. The MDN is powerful, but it may also be difficult to interpret. SubRep also lacks a direct self-continuity guarantee like MetaMo's self-model map. It can certify that skills help and that the library remains stable, but it does not fully certify that the agent remains recognizable to itself while learning.

SubRep's EASA and Waveformer/MORK integration are also still proposals, not demonstrated systems. The architecture is exciting, but it has not yet been empirically validated. So, the honest conclusion is that SubRep gives the strongest formal and architectural synthesis, but it is also the least proven in practice.

Overall, the four-paper progression can be summarized clearly. OpenPsi is biologically inspired and experimentally tested, but small and hand-coded. MetaMo gives a strong stability theory, but it is mostly descriptive and not adaptive. The bridge paper operationalizes MetaMo inside a Hyperon-style cognitive architecture, but many structures remain manually designed. SubRep provides the most complete formal planning and learning framework, including predictive coding, MDN learning, CDS/PDS admission, EASA, and conservative attention, but it still needs implementation and experimental proof.